Project Report:
Waste Disposal Manager System

Project Title : Waste Disposal Manager System

Course : CSA2001

Student Name : Anvesha Mudgal

Roll Number : 25BAI10607

Contents

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-functional Requirements
5. System Architecture
6. Design Diagrams
7. Design Decisions & Rationale
8. Implementation Details
9. Screenshots / Results
10. Testing Approach
11. Challenges Faced
12. Learnings & Key Takeaways
13. Future Enhancements
14. References

1. Introduction

The Waste Disposal Manager is a foundational command-line application designed to streamline the management and tracking of waste disposal requests. In many contexts, from small municipalities to large facilities, effective waste handling requires clear tracking of requests, waste types, weights, and current status. This project addresses the fundamental need for a simple, digital register for waste management data. Built using Python, it leverages object-oriented principles to model the core entities—Waste Request and Waste Manager—providing a robust yet straightforward solution for data manipulation and statistical reporting.

2. Problem Statement

The challenge is to replace inefficient, manual processes (like spreadsheets or physical logs) for tracking waste disposal requests with a centralized, scalable, and digital solution. Specifically, the system must allow staff to:

1. Digitally log new waste disposal requests efficiently.
2. Update the status of existing requests in real-time (e.g., Pending, Scheduled, Completed).
3. Provide immediate, actionable statistics, such as total weight processed and distribution by waste type.

This system aims to improve transparency, accountability, and operational efficiency in the waste management workflow.

3. Functional Requirements

ID	Requirement	Description
FR01	Add Waste Request	Users must be able to create a new Waste Request record, including details like waste type, weight (in kg), and location.
FR02	Update Request Status	Users must be able to modify the status of an existing request (e.g., from 'Pending' to 'Scheduled' or 'Completed').
FR03	View All Requests	The system must be able to display a list of all current waste requests with their full details.
FR04	View Request Details	Users must be able to retrieve and view the specific details of a request using its unique ID.
FR05	Generate Statistics	The system must provide aggregate statistics, including total weight of all waste, and a breakdown of total weight per waste type (e.g., 'Recyclable', 'Hazardous', 'General').

FR06	Data Persistence (Implied)	The system should save and load data so that requests persist between sessions (implemented via file I/O or a simple database).

4. Non-functional Requirements

ID	Requirement	Description
NFR01	Performance	Retrieval of all requests and calculation of statistics should occur in near real-time (sub-second response time).
NFR02	Usability	The command-line interface (CLI) must be intuitive, featuring a clear menu and validated user input.
NFR03	Maintainability	The code base must be modular, adhering to Python's object-oriented standards, using clear variable names and comments.
NFR04	Reliability	The system must handle common input errors (e.g., non-numeric input for weight) gracefully without crashing.
NFR05	Security	As a basic utility, robust security is not a primary concern, but sensitive data should be avoided, and file storage access should be restricted to the application only.

5. System Architecture

The WDM system follows a simple, two-tier architecture:

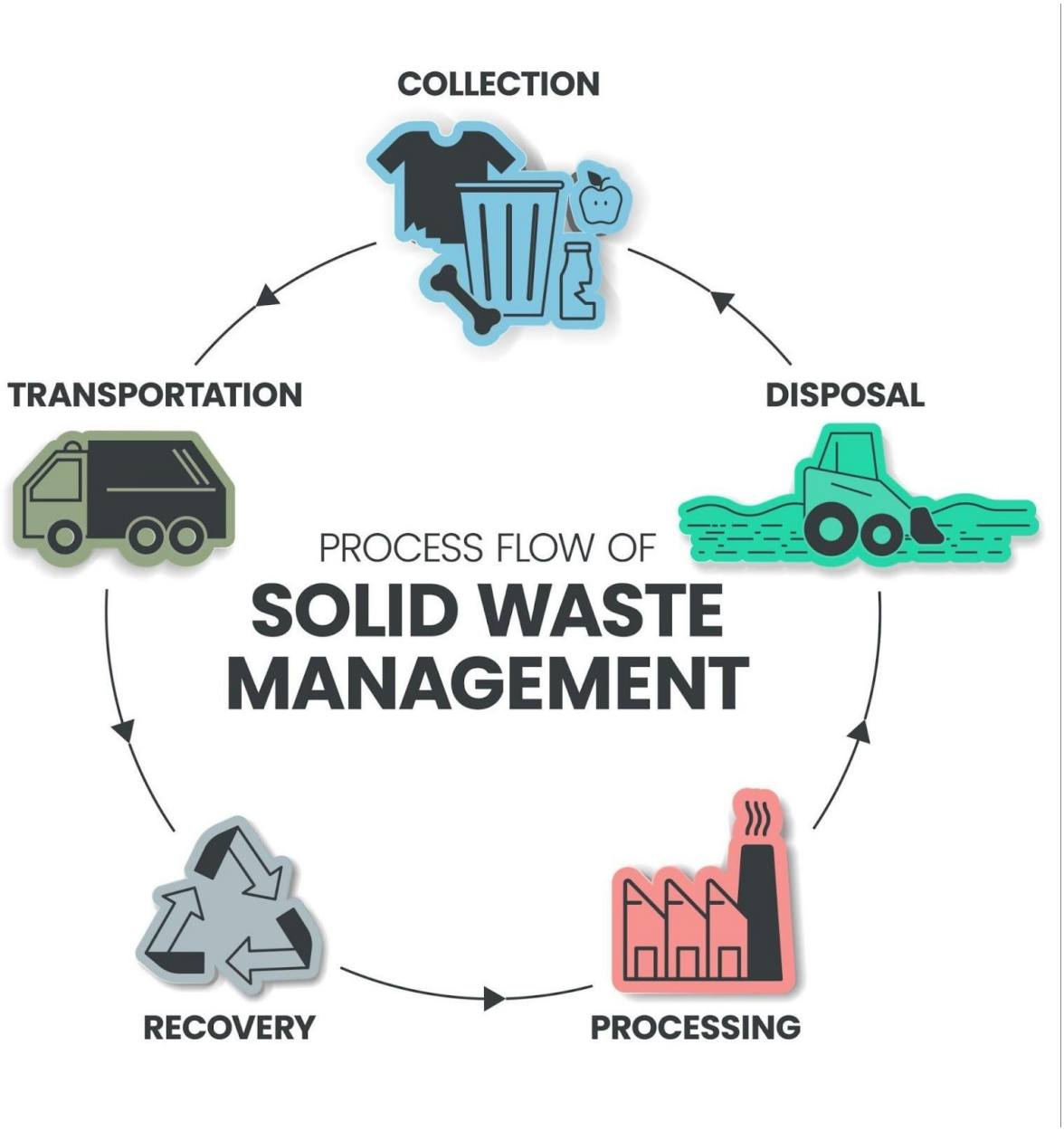
1. **Presentation/Application Layer (CLI):** This layer handles user interaction via the command line. It validates input and calls methods in the Manager Layer based on user selection.
2. **Manager/Data Layer:** This layer contains the core business logic.
   - **WasteRequest Class:** Data structure representing a single request.
   - **WasteManager Class:** Manages a collection of WasteRequest objects. It is responsible for CRUD operations (Create, Read, Update, Delete) and generating statistical reports.
   - **Data Storage:** A simple mechanism (e.g., a file or in-memory dictionary) is used for persistence.

The architecture emphasizes separation of concerns, where the WasteManager is the single point of control for all data manipulation.

6. Design Diagrams

Use Case Diagram

The use case diagram outlines the primary interactions the user (staff member) has with the system.



Workflow Diagram (Waste Request Lifecycle)

This diagram visualizes the status progression of a single waste request.

Sequence Diagram (Add New Request)

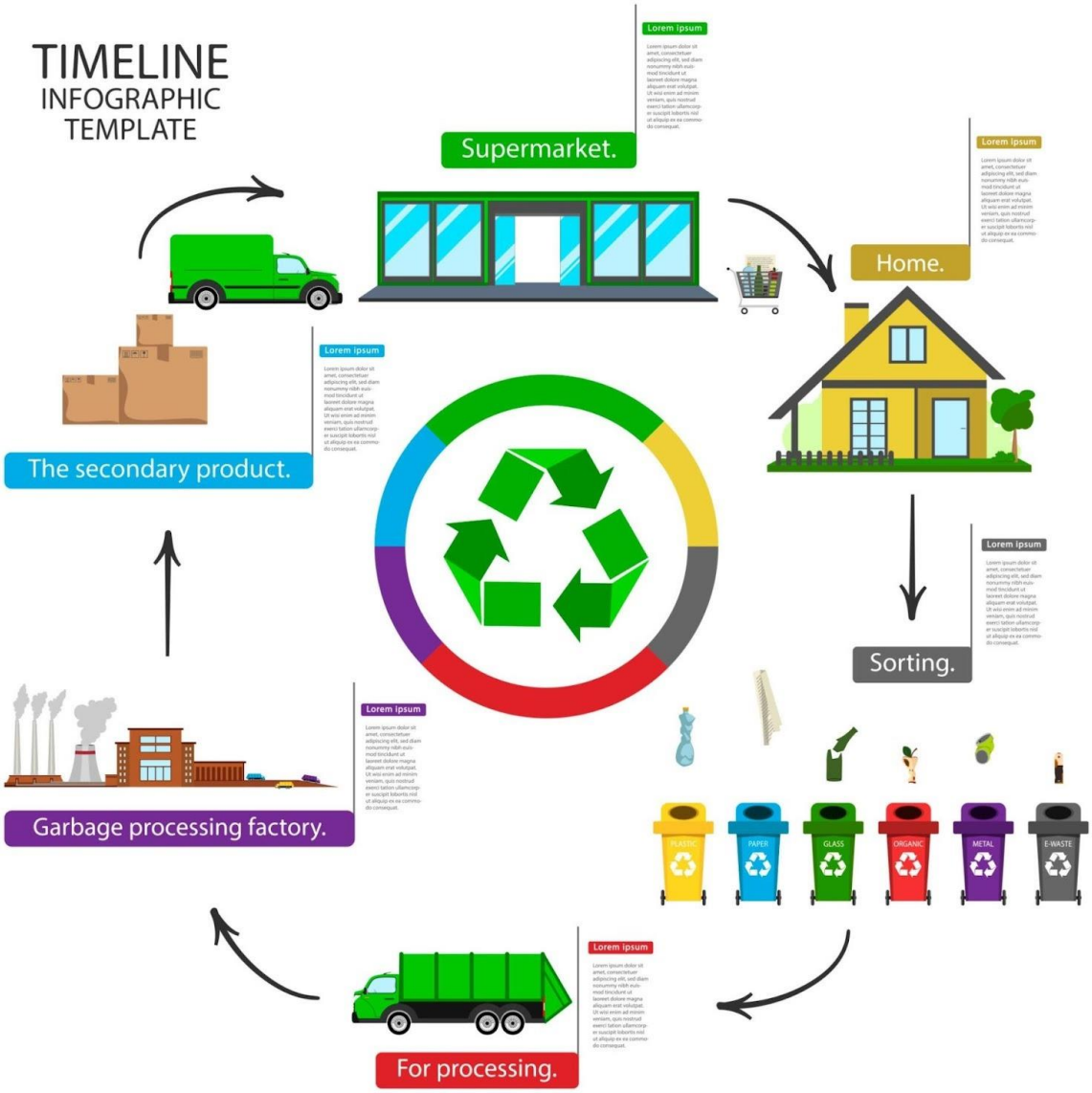
This diagram shows the sequence of steps and messages exchanged when a user adds a new waste request.

Class/Component Diagram

This diagram illustrates the core classes, their attributes, and their relationship within the system.

ER Diagram (Conceptual Data Model)

Assuming simple persistence (file or light database), this conceptual ER Diagram shows the structure of the data records.



7. Design Decisions & Rationale

Decision	Rationale
Python OOP Approach	Python was chosen for rapid development and readability. The use of classes (Waste Request and Waste Manager) enforces clear object-oriented principles, making the system easy to scale and maintain.
Unique ID Generation	Each Waste Request is assigned a unique, auto-incrementing ID. This is crucial for precise tracking, updating, and referencing individual requests.

In-Memory/File Storage	For simplicity, initial data storage is assumed to be in-memory (using a list/dictionary within Waste Manager) with optional simple file I/O for persistence. This avoids external database complexity for a basic utility.
Enumerated Status	Using a predefined set of statuses (Pending, Scheduled, Completed) ensures data consistency and simplifies status updates, preventing human error from free-text entries.
Centralized Logic (Waste Manager)	All business logic (statistics, CRUD) is encapsulated within the Waste Manager. This follows the Single Responsibility Principle and simplifies the CLI's role to just input/output handling.

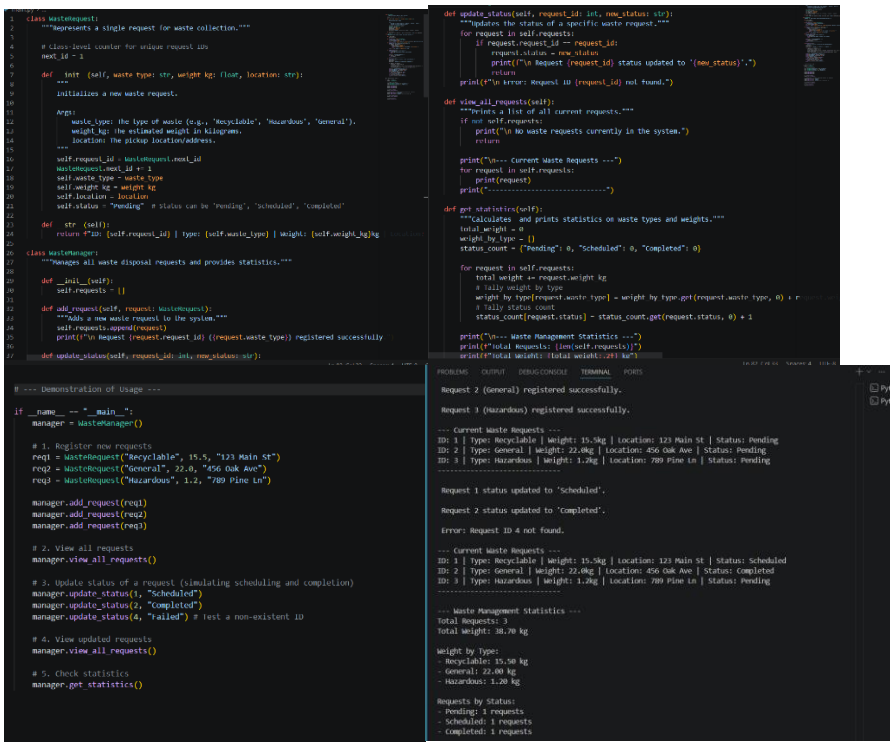
8. Implementation Details

The implementation is centered around the two Python classes:

1. **Waste Request:**
   - **Attributes:** request id, waste type (String), weight kg (Float), status (String/Enum, default 'Pending'), timestamp.
   - **Methods:** __init__, update_status, __str__ (for easy printing).
2. **WasteManager:**
   - **Attributes:** requests (a dictionary or list to store WasteRequest objects).
   - **Methods:**
     - add_request(waste_type, weight, location): Creates and stores a new WasteRequest.
     - update_request_status(id, new_status): Finds a request by ID and updates its status.
     - get_all_requests(): Returns the full list of requests.
     - get_statistics(): Calculates and returns total weight and weight breakdown by waste type.

The main script (mainsprog) provides a simple menu loop that takes user input and calls the corresponding WasteManager methods. Input validation is handled in the application layer before calling the manager.

9. Screenshots / Results



10. Testing Approach

The testing approach for the WDM system is based on Unit Testing and Functional Testing.

1. **Unit Testing:**
   - Testing the Waste Request class constructor to ensure attributes are set correctly.
   - Testing the update_status method in WasteRequest with valid and invalid status inputs.
   - Testing all CRUD methods in the WasteManager (e.g., ensuring a request is actually added, retrieved, and updated).
2. **Functional Testing:**
   - **Statistics Test:** Adding three distinct requests and verifying that get_statistics() correctly calculates the total weight and the breakdown percentages.
   - **Edge Case Test:** Attempting to update a non-existent request ID to ensure the system returns an appropriate error message and does not crash.
   - **Input Validation Test:** Entering non-numeric data for weight during request creation and verifying the system prompts for re-entry or gracefully handles the exception.

11. Challenges Faced

1. **Input Validation Complexity:** A primary challenge was robustly validating user input from the CLI, especially for required fields like weight_kg. Ensuring correct data types (e.g., float) and handling exceptions without breaking the application required careful error handling (try-except blocks) in the main program loop.
2. **ID Management:** Implementing a reliable, persistent unique ID system proved tricky with simple file I/O. A strategy was required to ensure the next ID was correctly calculated upon loading existing data to avoid duplicates.

3. **Data Structure Choice:** Initially, a simple list was considered for storing requests, but a dictionary using the request_id as the key was ultimately chosen in the WasteManager to provide O(1) time complexity for lookups and updates, which is essential for performance.

12. Learnings & Key Takeaways

1. **Importance of Encapsulation:** By centralizing all data logic within the WasteManager class, the project strongly reinforced the benefits of encapsulation and clear separation of concerns, making the code much easier to read and debug.
2. **Effective Use of Python Decorators:** Using methods like @property could further enhance the class design by managing attribute access, which is a valuable Python-specific learning.
3. **Designing for Scalability:** Even in a small project, designing with classes allowed for easy future expansion. For instance, adding a new feature like 'assigned collector' would only require modifying the WasteRequest class and the relevant WasteManager method, rather than restructuring the entire codebase.

13. Future Enhancements

1. **Database Integration:** Migrate from file I/O to a proper database (e.g., SQLite or PostgreSQL) for more robust, scalable, and concurrent data persistence.
2. **Web/GUI Interface:** Develop a simple web interface (using Flask/Django) or a graphical desktop application (using Tkinter/PyQt) to replace the basic command-line interface, improving usability.
3. **Advanced Reporting:** Implement features to filter requests by date range, location, or collector assigned, and export statistics into report formats (e.g., CSV or PDF).
4. **User Roles & Authentication:** Introduce separate user roles (e.g., Manager, Collector, Requestor) with corresponding access controls.
5. **Location Tracking:** Integrate coordinates or a more detailed address system for better collection route planning.

14. References

1. **Project Source Repository:**
   - mbharadwaj630-pixel/waste-disposal-manager. (n.d.). *GitHub*. Retrieved from https://github.com/mbharadwaj630-pixel/waste-disposal-manager
2. **Programming Language Documentation:**
   - The Python Software Foundation. (n.d.). *Python Documentation*. Retrieved from https://docs.python.org/
3. **Design Methodology:**

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

THANKYOU